```
  def handleEvents(state: State, event: Event): State = event match {
    case Opened(betId, walletId, marketId, odds, stake, result) =>
      OpenState(
        Status(betId, walletId, marketId, odds, stake, result),
        None,
        None)
    case MarketConfirmed(state) =>
      state.copy(marketConfirmed = Some(true))
    case FundsGranted(state) =>
      state.copy(fundsConfirmed = Some(true))
    case ValidationsPassed(state) =>
      state
    case Closed =>
      ClosedState(state.status)
    case Settled(_) =>
      SettledState(state.status)
    case Cancelled(betId, reason) =>
      CancelledState(state.status)
    case Failed(_, reason) =>
      FailedState(state.status, reason)
  ...
}
```

Note that `MarketConfirmed` and `FundsGranted` update the `Status` but do not change the `State`. `ValidationsPassed` refers to the fact that the timeout has expired. When all validations have passed, this event is generated.

Handling commands: Validating the market and the funds

Let's finish with the rest of the command handler methods. When the bet queries the market status (`requestMarketStatus`), it must get a quick answer from the market. Let's say that according to business requirements, you cannot use market information more than three seconds old. There is a financial risk, and business experts have requested this feature.